

Révision

Lecture du capteur LM35 avec ADC (PIC18F4550)

Le capteur LM35 délivre :

- 10 mV / °C
- ADC 10 bits → valeur de 0 à 1023
- Vref = 5V

Formule : $\text{Température} = (\text{Valeur_ADC} \times 5 \times 100) / 1023$

Code MPLAB (XC8) :

```

1  #include <xc.h>
2  #define _XTAL_FREQ 8000000
3
4  // Configuration bits (exemple)
5  #pragma config FOSC = HS
6  #pragma config WDT = OFF
7  #pragma config LVP = OFF
8
9  // Initialisation ADC
10 void ADC_Init()
11 {
12     ADCON1 = 0x0E; // AN0 analogique, reste digital
13     ADCON2 = 0xA9; // Justification droite, Tacq, Fosc/8
14     ADCON0 = 0x01; // Activer ADC sur AN0
15 }
16
17 // Lecture ADC
18 int ADC_Read()
19 {
20     GO_NDONE = 1;
21     while(GO_NDONE); // attendre fin conversion
22     return ((ADRESH<<8) + ADRESL);
23 }
24
25 void main()
26 {
27     int adc_value;
28     float temperature;
29
30     TRISA = 0x01; // RA0 entrée
31
32     ADC_Init();
33
34     while(1)
35     {
36         adc_value = ADC_Read();
37
38         // Conversion en °C
39         temperature = (adc_value * 5.0 * 100.0) / 1023.0;
40
41         __delay_ms(1000);
42     }
43 }

```

Ce programme configure la broche AN0 comme entrée analogique et initialise le module ADC interne du microcontrôleur afin de lire la valeur fournie par le capteur LM35. Il convertit ensuite la tension mesurée en une valeur de température exprimée en degrés Celsius.

Énoncé du TP

Dans ce travail pratique, on vous demande de réaliser un système embarqué basé sur le microcontrôleur **PIC18F4550** permettant de concevoir un compteur numérique simple allant de **0 à 9**. Le programme doit piloter un afficheur **7 segments à cathode commune** connecté au PORTB du microcontrôleur. Le principe consiste à utiliser un tableau de correspondance contenant les codes binaires de chaque chiffre de 0 à 9, afin de transformer une valeur numérique en signal adapté à l'afficheur. Le microcontrôleur est programmé en langage C (XC8) pour incrémenter automatiquement un compteur dans une boucle infinie, avec un délai d'une seconde entre chaque incrémentation afin de rendre l'affichage visible. Ainsi, le système affiche successivement les chiffres de 0 à 9 de manière cyclique, démontrant le fonctionnement du microcontrôleur, la gestion des ports d'E/S et le pilotage d'un afficheur 7 segments.

```

Source History
1 #include <xc.h> // Bibliothèque principale du PIC (accès registres)
2
3 #pragma config FOSC = HS // Utilisation d'un oscillateur externe haute vitesse
4 #pragma config WDT = OFF // Désactivation du Watchdog Timer
5 #pragma config LVP = OFF // Désactivation de la programmation basse tension
6
7 #define _XTAL_FREQ 8000000 // Définition de la fréquence du quartz (8 MHz)
8
9 // Tableau contenant les codes pour afficher les chiffres 0 à 9 sur 7 segments
10 unsigned char seg_code[10] = {
11     0x3F, // 0 → segments a,b,c,d,e,f allumés
12     0x06, // 1 → segments b,c
13     0x5B, // 2 → a,b,d,e,g
14     0x4F, // 3 → a,b,c,d,g
15     0x66, // 4 → b,c,f,g
16     0x6D, // 5 → a,c,d,f,g
17     0x7D, // 6 → a,c,d,e,f,g
18     0x07, // 7 → a,b,c
19     0x7F, // 8 → tous les segments
20     0x6F, // 9 → a,b,c,d,f,g
21 };
22
23 void main() { // Fonction principale du programme
24
25     int i = 0; // Variable compteur
26
27     TRISB = 0x00; // Configure PORTB en sortie (connecté au 7 segments)
28
29     while(1) { // Boucle infinie (programme tourne en continu)
30
31         for(i = 0; i < 10; i++) { // Boucle de 0 à 9
32
33             LATB = seg_code[i]; // Envoie le code du chiffre vers le 7 segments
34
35             __delay_ms(1000); // Attente de 1 seconde entre chaque chiffre
36         }
37     }
38 }

```